Bittensor 技术分享

去中心化 AI 能力市场

2026.3.20 — 黄仁勋说了什么?

> *"现代版 Folding@home"*
> *— NVIDIA CEO 黄仁勋，评价 Covenant-72B（SN3 Templar）*

- **去中心化训练**：70+ 无许可节点，无数据中心，仅靠普通商用互联网完成 **72B** 模型预训练
- **MMLU 67.1**：对标 Llama-2-70B，去中心化训练首次达到主流开源模型水准
- **市场验证**：SN3 alpha 一月涨 **444%**，TAO 峰值 **$377**

Covenant-72B：史上最大去中心化 LLM 预训练

指标	数值
参数量	**72B**
训练数据	~1.1 万亿 token
参与节点	**70+，无许可**
MMLU 得分	**67.1**（对标 Llama-2-70B）
基础设施	普通商用互联网，无数据中心

> ***这是怎么做到的？Bittensor 如何激励 70 个陌生节点协同训练？***

Part 1 · Bittensor 是什么?

核心定位

> *比特币激励全球矿工**维护账本***
> *Bittensor 激励全球 GPU 节点**竞争产出最优 AI 智能***

- **不是**出租算力（≠ Akash / Render）
- 而是**评估 AI 输出质量**，奖励最优输出的参与者
- 每个子网自定义 AI 任务：推理 / 训练 / embedding / 图像…

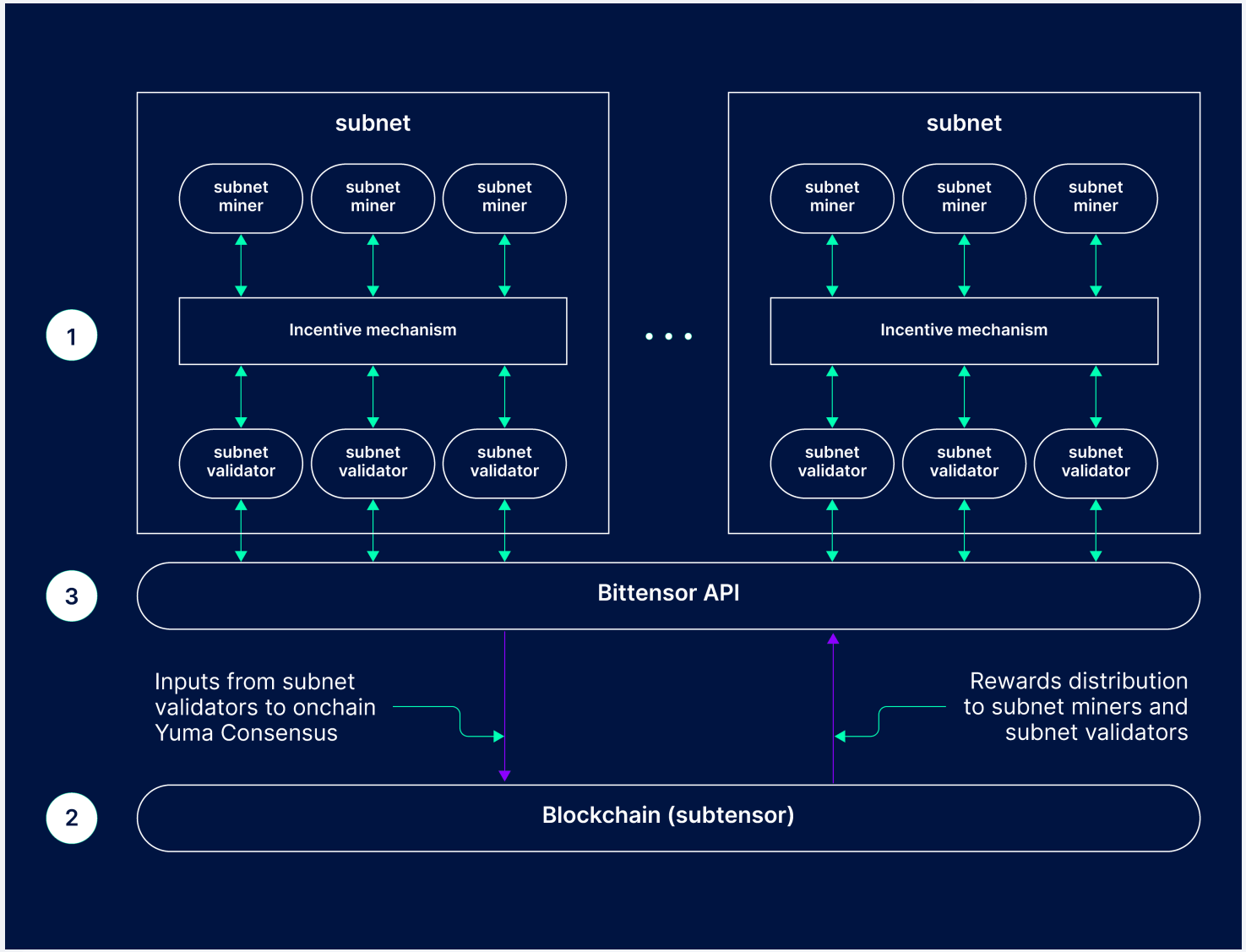
和以太坊的区别

维度	以太坊	Bittensor
解决的问题	去中心化通用计算	去中心化 AI 能力市场
节点工作	执行 EVM 字节码	运行 AI 模型，竞争最优输出
共识目标	对"状态转移"达成一致	对"AI 输出质量"达成一致
激励对象	区块验证者（质押 ETH）	Miner/Validator（质量越高收益越多）
链技术	EVM / Solidity	Substrate / Rust

关键差异：Bittensor 本质是一个链上 AI 质量排名系统

Part 2 · 网络架构

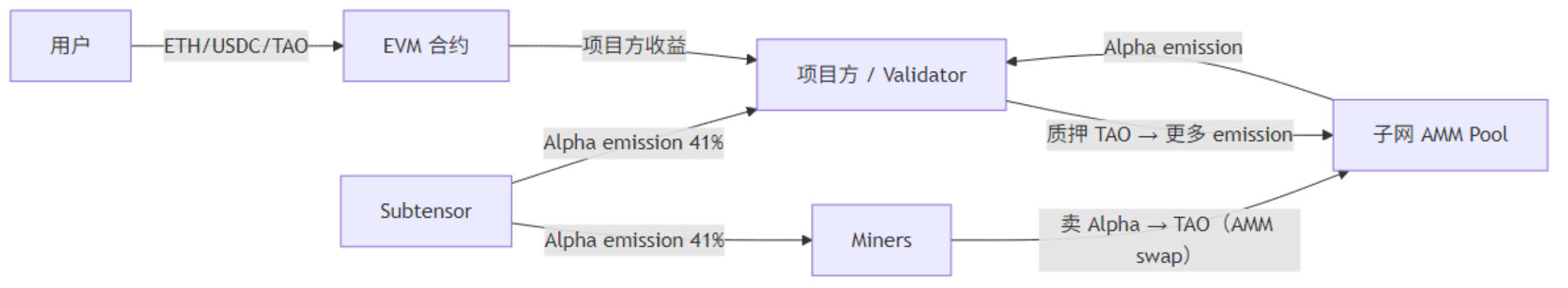
三层结构


subnet
subnet miner
subnet miner
subnet miner
Incentive mechanism
subnet validator
subnet validator
subnet validator
subnet
subnet miner
subnet miner
subnet miner
Incentive mechanism
subnet validator
subnet validator
subnet validator
1
3
Bittensor API
Inputs from subnet validators to onchain Yuma Consensus
Rewards distribution to subnet miners and subnet validators
2
Blockchain (subtensor)

角色与收益

角色	职责	Alpha Emission
Miner	运行 AI 任务，通过 Axon 暴露服务	**41%**
Validator	查询 Miner，评分，set_weights	**41%**
Subnet Owner	定义任务规则，维护子网	**18%**

资金流

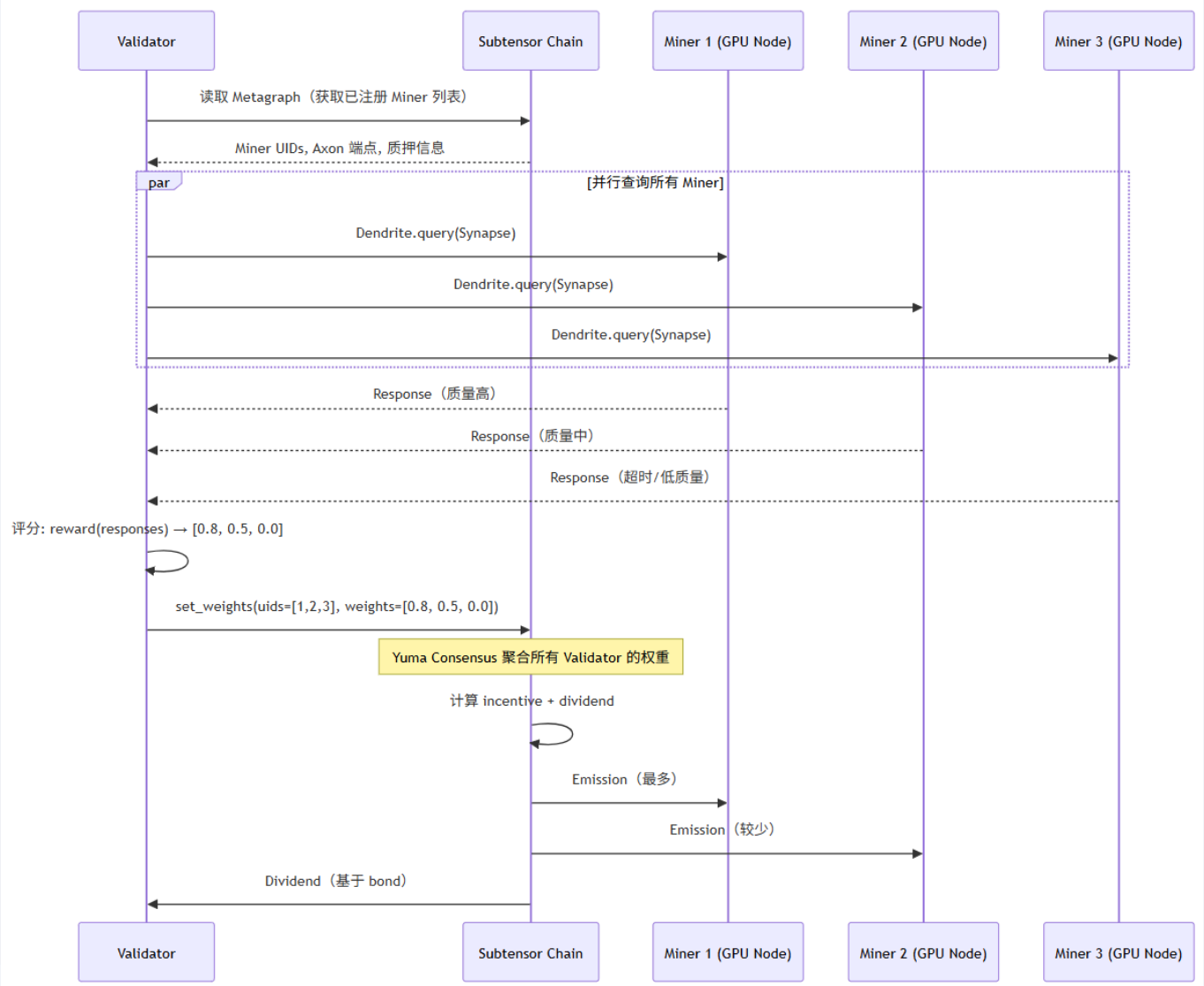


SDK 命名来源：神经科学隐喻

术语	对应概念
Axon	Miner 的服务端点（监听端口）
Dendrite	Validator 的请求客户端
Synapse	请求 / 响应的消息结构体
Metagraph	子网所有节点状态的链上快照

Part 3 · 核心流程

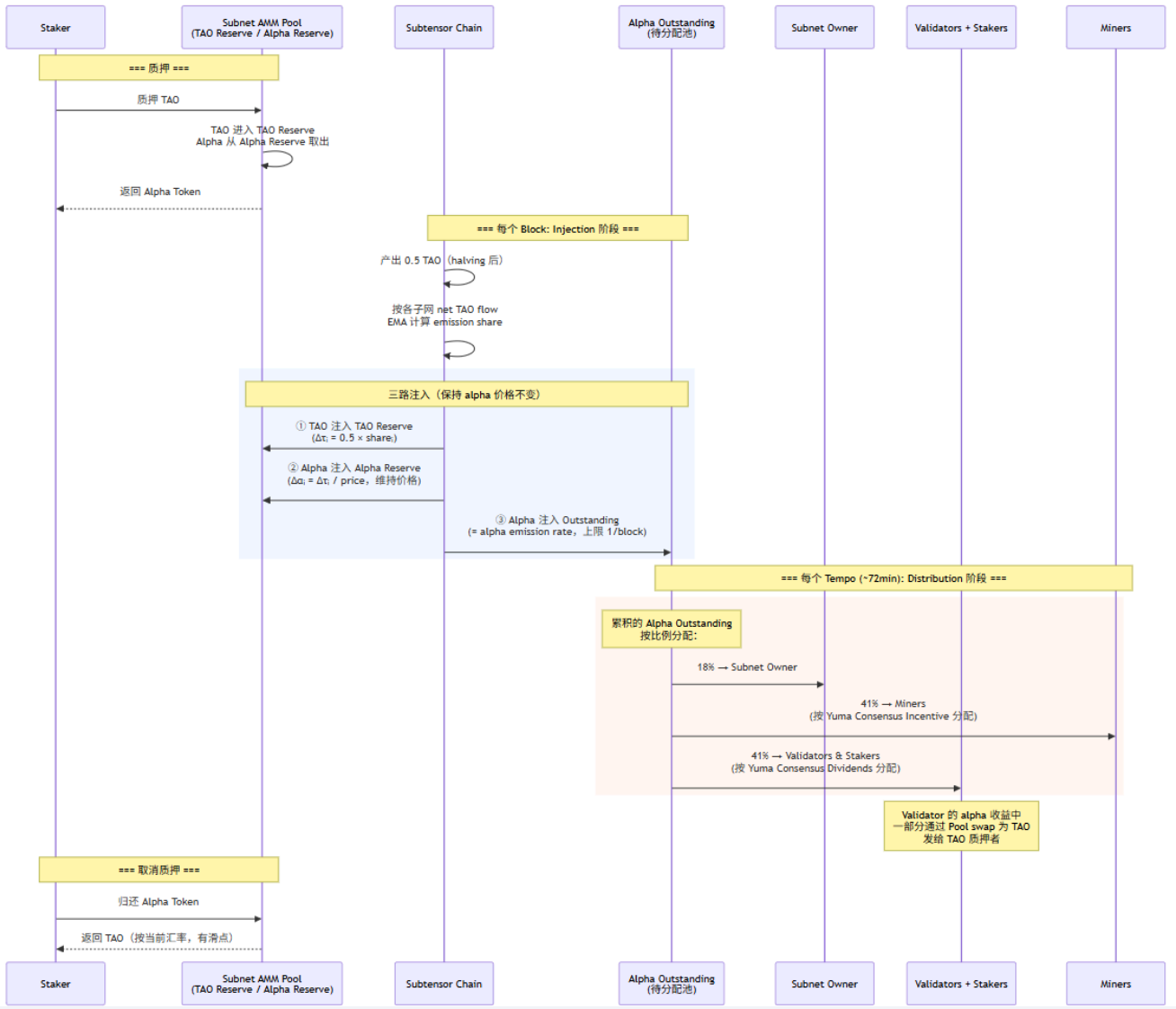
Miner-Validator 交互



每个 block 产出的 0.5 TAO 如何在子网间分配?

```
1. 跟踪每个子网的 net TAO flow:
   net_flow = Σ(TAO staked) - Σ(TAO unstaked)

2. 计算 EMA（86.8 天窗口，30 天半衰期）:
   S_i = (1-α) × S_{i-1} + α × net_flow_i    (α ≈ 0.000003209)

3. 计算动态下界 L（相对基准）:
   L = max(FlowCutoff, min_j( min(S_j, 0) ))
   |    └── 治理参数，防止 L 过低    └── 所有子网 EMA 中最负的那个（正值取 0）

4. 裁剪 + 归一化:
   z_i = max(S_i - L, 0)          把最差子网的 z_i 归零，其余子网按相对优势分配 emission
   share_i = z_i / Σ z_j          （线性分配）

4. 最终注入:
   Δτ_i = 0.5 TAO × share_i
```

dTAO 质押与 Emission 流程



Emission ① Injection（每 Block ~12s）

```
0.5 TAO 按各子网净流入 EMA 分配 → 三路注入：

  ① TAO Reserve  += Δτ          增加流动性

  ② Alpha Reserve += Δτ / p     维持价格不变

  ③ Alpha Outstanding += α      待分配给参与者（上限 1/block）
```

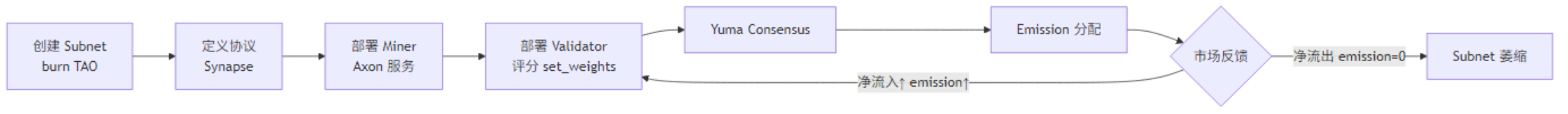
- ① 和 ② 增加池的流动性（降低交易滑点），但**不改变 alpha 价格**
- ③ 是真正要分给参与者的 Alpha

Emission ② Distribution（每 Tempo ~72min）

```
每个 tempo (~360 blocks) 结算：

Alpha Outstanding 累积量按比例分配：
├── 18% → Subnet Owner（收到 Alpha）
├── 41% → Miners（收到 Alpha，按 Incentive 分配）
└── 41% → Validators & Stakers
    ├── Validator 抽取佣金（Alpha）
    └── 剩余分给 Stakers：
        ├── Alpha Stakers → 收到 Alpha
        └── TAO Stakers → Alpha 通过池 swap 为 TAO 后发放
```

> *Miner/Validator 收到的是 **Alpha Token**，TAO Staker 份额通过 AMM 自动 swap 为 TAO*

Subnet 生命周期



- 净流入多 → 更多 emission; 净流出 → emission 归零
- 拥有 subnet 方式：创建，购买
- subnet 总量：256，大于时这个数将会末位淘汰

Part 4 • Yuma Consensus

如何防止 Validator 互相勾结?

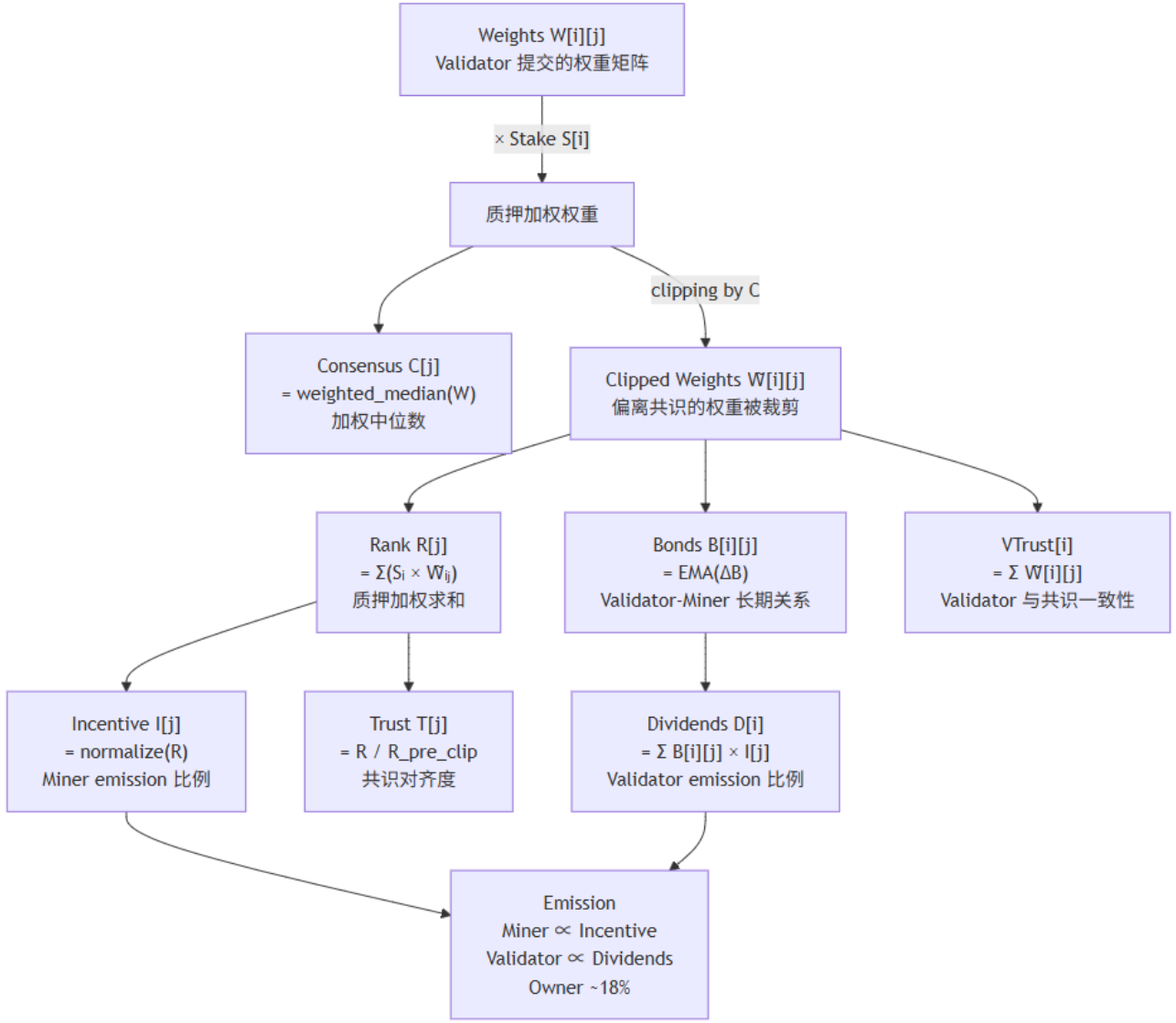
为什么需要 Yuma Consensus?

攻击方式	Yuma 的防护
少数大户单独操纵评分	加权**中位数**，非均值
给偏向 Miner 打虚高分	**Clipping**：高于共识的分被截断
频繁切换投票获利	**Bond EMA**：长期稳定评分才有高收益
验证质量低劣	**VTrust**：被采纳权重占比衡量诚实度

算法六步总览

```
输入：W[i][j] = Validator i 对 Miner j 的评分，S[i] = 质押量


Step 1  质押加权      Validator 影响力 ∝ S[i]

Step 2  Consensus    C[j] = weighted_median(W[:,j], S)  ← 加权中位数，抵抗极端评分

Step 3  Clipping     W̃[i][j] = min(W[i][j], C[j])      ← 只截高于共识

Step 4  Rank         R[j] = Σᵢ S[i] × W̃[i][j]

Step 5  Incentive    I[j] = R[j] / ΣR                  ← Miner emission

Step 6  Trust        T[j] = R[j] / R_pre_clip

Step 7  Bond         B[i][j] = EMA(ΔB[i][j])           ← 长期关系

Step 8  Dividends    D[i] = Σⱼ B[i][j] × I[j]          ← Validator emission
```

参数推导关系图


Weights W[i][j]
Validator 提交的权重矩阵
× Stake S[i]
质押加权权重
clipping by C
Consensus C[j]
= weighted_median(W)
加权中位数
Clipped Weights W̃[i][j]
偏离共识的权重被裁剪
Rank R[j]
= Σ(Sᵢ × W̃ᵢⱼ)
质押加权求和
Bonds B[i][j]
= EMA(ΔB)
Validator-Miner 长期关系
VTrust[i]
= Σ W̃[i][j]
Validator 与共识一致性
Incentive I[j]
= normalize(R)
Miner emission 比例
Trust T[j]
= R / R_pre_clip
共识对齐度
Dividends D[i]
= Σ B[i][j] × I[j]
Validator emission 比例
Emission
Miner ∝ Incentive
Validator ∝ Dividends
Owner ~18%

示例：场景设定

2 个 Validator，3 个 Miner

	V1	V2
Stake	**100 TAO**	**60 TAO**
对 M0 评分	0.7	0.4
对 M1 评分	0.2	**0.5** ← 偷高分
对 M2 评分	0.1	0.1

> *“ V1 持有 62.5% stake（> 50%），V1 的评分即为加权中位数 ”*

Step 2–3：加权中位数 & Clipping

加权中位数（升序，累计权重首次 ≥ 总权重/2 时停止）

Miner	排列	累计权重	中位数 C
M0	[(0.4,V2=60),(0.7,V1=100)]	60→160	**0.7**
M1	[(0.2,V1=100),(0.5,V2=60)]	100 ≥ 80 停	**0.2**
M2	[(0.1,V1=100)]	100 ≥ 80 停	**0.1**

Clipping： `W̃[i][j] = min(W[i][j], C[j])`

| V2 对 M0: 0.4 < C=0.7 → 保留 0.4 | V2 对 M1: **0.5 > C=0.2 → 截断为 0.2** |

Step 4–6：Rank / Incentive / Trust

Miner	Rank 计算	Rank	Incentive
M0	100×0.7 + 60×0.4	**94**	**0.662**
M1	100×0.2 + 60×0.2	**32**	**0.225**
M2	100×0.1 + 60×0.1	**16**	**0.113**

Trust（Pre-clip M1 = 100×0.2 + 60×0.5 = 50）

| M0: 94/94 = **1.00** | M1: 32/50 = **0.64** ← V2 给了偏高分 | M2: **1.00** |

Emission 分配：偷分没有收益

假设该 tempo 子网分得 **100 Alpha**

接收方	Incentive/Dividends	Alpha
M0	41% × 0.662	**27.1**
M1	41% × 0.225	**9.2**
M2	41% × 0.113	**4.6**
V1	41% × 0.705	**28.9**
V2	41% × 0.295	**12.1 ← 偷分无效**
Owner	18%	18.0

> *“ V2 对 M1 打 0.5 → 被截到 0.2，Dividends 仅 12.1 < V1 的 28.9 ”*

Part 5 · 经济学（dTAO）

TAO 参数 & AMM 池

参数	值
最大供应量	21,000,000 TAO（同 BTC）
出块时间	~12 秒
当前区块奖励	**0.5 TAO**（2025.12 首次 halving 后）
每日 emission	~3,600 TAO/天

每个子网有独立 AMM 池（恒定乘积，无手续费）

```
质押 TAO    → TAO 进 TAO Reserve，Alpha 从 Alpha Reserve 取出 → 用户得 Alpha

取消质押    → 归还 Alpha → TAO 从 TAO Reserve 取出 → 用户得 TAO（有滑点）

Alpha 价格  = TAO Reserve / Alpha Reserve
```

质押 TAO = 为子网"投票"

```
子网 emission 份额 ∝ 净 TAO 流入的 EMA（86.8 天窗口）

净流入多 → 获得更多 emission → Alpha 预期上涨
净流出   → emission 归零   → Alpha 卖压增大
```

市场机制：emission 自动流向最受资金认可的子网，无需人工治理。

Alpha 价格为什么普遍很低?

Miner/Validator 收到 Alpha 后**持续通过 AMM 卖回 TAO** → 形成持续卖压

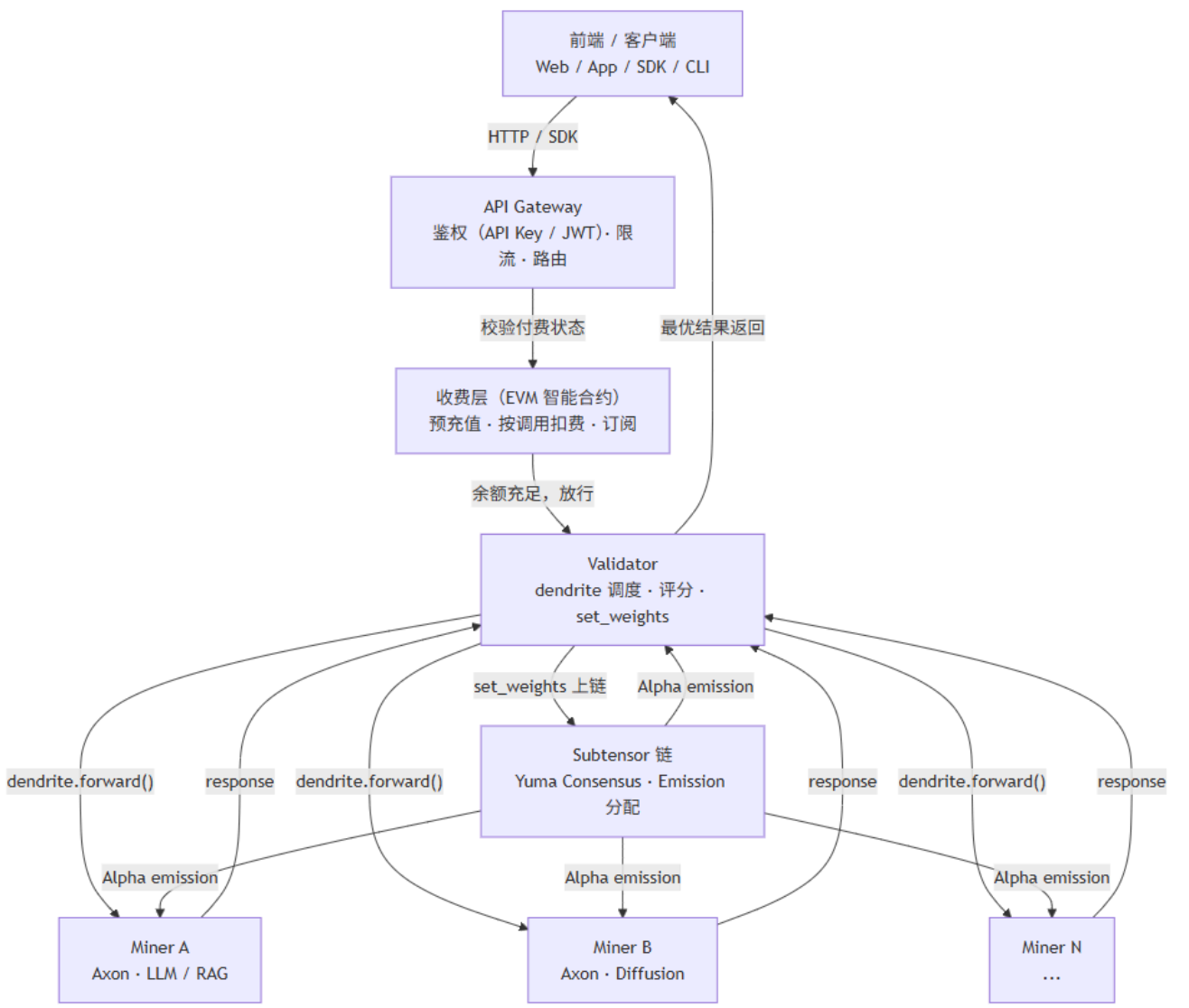
Alpha 价格反映的是「**市场对该子网未来净流入的预期**」，而非当前质押量。

典型数据（Subnet 1）：

```
TAO Reserve:   657 τ     Alpha Reserve: 161k α

Alpha 价格:    0.0004 τ/α（1 TAO ≈ 2500 Alpha）

EMA 净流入:    -0.0338（净流出 → emission 为零）
```

Part 6 · 商业化场景

整体调用链架构


前端 / 客户端
Web / App / SDK / CLI
HTTP / SDK
API Gateway
鉴权（API Key / JWT）· 限流 · 路由
校验付费状态
最优结果返回
收费层（EVM 智能合约）
预充值 · 按调用扣费 · 订阅
余额充足，放行
Validator
dendrite 调度 · 评分 · set_weights
set_weights 上链
Alpha emission
Subtensor 链
Yuma Consensus · Emission 分配
dendrite.forward()
response
dendrite.forward()
response
dendrite.forward()
response
Alpha emission
Alpha emission
Alpha emission
Miner A
Axon · LLM / RAG
Miner B
Axon · Diffusion
Miner N
...

三种收费模型

模式	流程	特点
预付费 Credits	用户充值 → 合约记余额 → 每次调用扣减	类似 OpenAI credits，体验好
按调用付费	用户签名 → 合约验证 → 扣费 → 放行	精细计费（token / request）
订阅制	持有 NFT / token → API 验证持仓	Web3-native，适合 SaaS

Web2 vs Bittensor 架构对比

模块	Web2（OpenAI）	Bittensor 方案
API	OpenAI API	自建 API Gateway
模型	自有	去中心化 Miners
调度	内部黑盒	Validator（可审计）
收费	Stripe	EVM 合约
激励	公司利润	Emission + Fee
抗审查	无	无许可，全球节点

Part 7 • Demo

Demo A：直接调用现有子网 API

SN64 Chutes（OpenAI 兼容接口）

```
pip install chutes openai
chutes register
chutes keys create --name demo-key --admin   # → cpk_xxx
```

```
from openai import OpenAI
client = OpenAI(api_key="cpk_xxx", base_url="https://llm.chutes.ai/v1")
resp = client.chat.completions.create(
    model="deepseek-ai/DeepSeek-V3-0324",
    messages=[{"role": "user", "content": "你好，请介绍一下自己"}],
    max_tokens=128,
)
print(resp.choices[0].message.content)
```

SN19 Nineteen： 申请 Key → `api_key="your_nineteen_key"` + `base_url="https://api.nineteen.ai/v1"`

Demo B：自建 Validator 提供 HTTP API

```
用户 curl/SDK
  |  POST /chat
  ▼
Validator (FastAPI :8000)
  |  dendrite.forward() → 直连 Miner IP:Port
  ▼
Miner (Axon :8091)   →  set_weights → Subtensor
```

```
btcli subnet register --netuid 461 \
  --wallet.name mywallet --wallet.hotkey myhotkey \
  --subtensor.network test

python miner.py --netuid 461
python validator.py --netuid 461

curl -X POST http://localhost:8000/chat \
  -H "Content-Type: application/json" \
  -d '{"prompt": "What is Bittensor?"}'
```

Part 8 · SDK 接口

三个核心文件

> *Sample Interface: github.com/latent-to/bittensor-subnet-template*

```
bittensor-subnet-template/
├── neurons/
|    ├── miner.py       ← 实现 forward()，处理 Validator 请求
|    └── validator.py   ← 查询 Miner，打分，set_weights()
└── template/
     └── protocol.py    ← 定义 Synapse（请求/响应结构）
```

```
pip install bittensor bittensor-cli
```

定义协议 & Miner 实现

```python
# protocol.py
class MyProtocol(bt.Synapse):
    query: str
    response: str = ""

# miner.py
def forward(synapse: MyProtocol) -> MyProtocol:
    synapse.response = my_model.generate(synapse.query)
    return synapse

axon = bt.Axon(wallet=wallet, port=8091)
axon.attach(forward_fn=forward)
axon.serve(netuid=NETUID, subtensor=subtensor)
axon.start()
```

Validator 实现

```
# validator.py
metagraph = subtensor.metagraph(netuid=NETUID)

responses = await dendrite.forward(
    axons=metagraph.axons,
    synapse=MyProtocol(query="Hello"),
    timeout=12.0,
)

scores = [1.0 if r.response else 0.0 for r in responses]
weights = normalize(scores)

subtensor.set_weights(
    netuid=NETUID,
    uids=metagraph.uids,
    weights=weights,
    wallet=wallet,
)
```

Part 9 · 技术深挖：SparseLoCo

Covenant-72B 是怎么做到的?

去中心化训练的带宽瓶颈

传统分布式训练（PyTorch DDP）

```
每步同步完整梯度 → 需要高带宽数据中心内部网络

数据中心：100 Gbps+

普通商用网络：100 Mbps ~ 1 Gbps
```

跨互联网训练 72B 模型 → 带宽差距 **100x**，直接梯度同步不可行

SparseLoCo：97% 压缩率

```
传统 DDP：每步同步完整梯度

SparseLoCo：
  本地迭代 15~250 步
      ↓
  只选 1~3% 核心梯度（稀疏选择）
      ↓
  量化为 2-bit（压缩率 97%）
      ↓
  通过 S3/R2 对象存储异步交换
      ↓
  其他节点合并更新，继续本地迭代
```

结果：普通商用网络即可参与 72B 模型训练

Part 10 · 挑战与风险

挑战与风险

技术

- 评估难题：创意类任务难以自动化评分
- 延迟：去中心化推理 > 中心化 API

经济

- Emission 集中：头部节点可能垄断
- 大量子网难以持续吸引参与者

生态

- 与 OpenAI / Anthropic / Google 的持续竞争
- 监管不确定性
- 子网开发学习曲线较陡

参考链接

文档 & 代码

资源	地址
官方文档	docs.bittensor.com
SDK	github.com/opentensor/bittensor
Subnet 模板	github.com/latent-to/bittensor-subnet-template
SN3 Templar	github.com/tplr-ai/templar
Covenant-72B 报告	templarresearch.substack.com

网络监控 & API

资源	地址
全网浏览器	taostats.io
SN19 Nineteen	nineteen.ai/app/api
SN64 Chutes	chutes.ai/docs/getting-started/quickstart